



# 语言分析技术实验报告

## 实验 3：C—语言的句法分析器

姓 名：田佳铭  
学 号：22920172204211  
院 系：信息学院人工智能系  
专 业：智能科学与技术  
年 级：2017 级  
指导教师：史晓东

2020 年 5 月 3 日

## 目录

|                          |           |
|--------------------------|-----------|
| 一、实验目的.....              | 3         |
| 二、实验环境及工具.....           | 3         |
| 三、实验与附件说明.....           | 3         |
| 四、设计思想.....              | 4         |
| 1.总体设计思路.....            | 错误！未定义书签。 |
| 2.训练集标注.....             | 错误！未定义书签。 |
| 3.hmm 与输入法实现.....        | 错误！未定义书签。 |
| 3.1hmm.....              | 错误！未定义书签。 |
| 3.2 最佳匹配实现.....          | 错误！未定义书签。 |
| 3.3 输入法实现逻辑（无 GUI） ..... | 错误！未定义书签。 |
| 4.GUI 设计 .....           | 错误！未定义书签。 |
| 5.实验结果.....              | 错误！未定义书签。 |
| 5.1 模型评估结果.....          | 错误！未定义书签。 |
| 5.2 无 GUI 界面的输出测试： ..... | 错误！未定义书签。 |
| 5.3GUI 界面示例.....         | 错误！未定义书签。 |
| 5.4 测试结果.....            | 错误！未定义书签。 |
| 6.现存的一些问题与可能的改进方案.....   | 错误！未定义书签。 |
| 五、遇到的问题与解决.....          | 16        |

## 一、实验目的

1. C--语言的句法分析器。读入一个 C--语言程序，判断该程序是不是一个合法的 C--语言程序。如果程序有语法和语义错误，请给出错误信息。至少处理以下错误：

(1) 所有语法错误。C--的语法参见 C--的文档；

(2) 语义错误：

2a: 标志符未定义错误，多次定义错误（需要采用符号表。关于符号表的实现，请大家参阅龙书的有关章节或我的 `symbol_table.ppt`。）

2b: 类型错误。判断运算符的参数是否具有所需要的类型。注意：算术运算符包括：+，-，\*，/，%只作用于整数。关系运算符（<,>，等）可以作用于整数表达式，字符表达式，但是不要求作用于字符串表达式（否则 C--就不是 C 语言的子集了，虽然理论上支持“ab” < “ac”还是挺有意思的）。类型错误包括函数调用的参数。

2. 程序要做成命令行程序，带一个参数，表示输入的 C--文件名。

## 二、实验环境及工具

本实验使用 flex 和 yacc 工具来编写。

先编写(\*)l 格式的文件，通过 flex 工具生成 lex.yy.c 文件；再编写(\*)y 格式的文件，通过 yacc 工具生成 y.tab.c 和 y.tab.h 文件；之后通过 GNU GCC 编译器生成可执行文件。

## 三、实验与附件说明

本次实验独立完成，参考了网上和老师的一些资料，没有和同学进行内容上的讨论。

附件中文件说明:

- (1) **project3.l**: 实验 3 的最终(\*)**.l** 文件
- (2) **project3.y**: 实验 3 的最终(\*)**.y** 文件
- (3) **lex.yy.c**: **project3.l** 生成的文件
- (4) **y.tab.c** 和 **y.tab.h**: **project3.y** 生成的文件
- (5) **句法规则框架.txt**: 本次实验使用的句法规则框架
- (6) **文件夹 S**: 存放各种测试文件

## 4.1 词法分析器设计

词法分析器的设计和实验一大致相同，主要的区别在于将命令行参数取消（因为只需要调用其中的 `yylex` 函数）和规则部分函数的修改。

### 4.1.1 声明部分

声明部分的正则定义部分设计如下:

|          |                                                                                |
|----------|--------------------------------------------------------------------------------|
| /        | *正规定义*/                                                                        |
| delim    | [ \t]                                                                          |
| ws       | {delim}+                                                                       |
| leter    | [A-Za-z]                                                                       |
| digit    | [0-9]                                                                          |
| star     | **                                                                             |
| id       | {leter}{(leter){digit}}*                                                       |
| number   | {digit}+(\.{digit})+?(E[+-]?{digit})?                                          |
| comment  | (\#(\^\\n)*)(\^\\/((\^)*(\^\\^\\V)+(\^)*(\^)*(\^\\V)*(\^\\V)))/(\^\\/(\^\\n)*) |
| string   | \(")\^"                                                                        |
| chartype | (\\.\\')(\\\\"\\')                                                             |
| special  | [\\@\\~\\\\\$\\%]                                                              |
| type     | (void) (int) (char) (long) (short)                                             |
| pointer  | {type}{delim}*{star}+                                                          |
| pun      | ("")[""]['']["{}"; "=" "*" "/ % "+" - ' ,"                                     |
| rel      | (<=) (>=) (<) (>) (!=) (==)                                                    |

图 4-1 词法分析器声明部分正则定义部分

其中实际用到的有（因为很多是直接拷贝实验 1 的定义，有些是不需要使用的）：

- (1) ws: 空串

- (2) id: 变量, 以字母开头, 由字母和数字组成
- (3) number: 数字, 包括整数与小数, 也包括正数与负数 (认为 001 这样的数字合法, 且认为只有 10 进制数)
- (4) comment: 注释
- (5) string: 字符串常量

### 4.1.2 规则部分

规则部分设计如下:

```
%%
"\n"      { nline++;}
{comment} /*nothing to do*/
{ws}      /*nothing to do*/
{number}  {yylval._ident = insert_id(yytext); return NUM;}
"__stdcall" {return STDCALL;}
"__cdecl"  {return CDECL;}
"if"       {return IF;}
"int"      {return INT;}
"void"     {return VOID;}
"char"     {return CHAR;}
"else"     {return ELSE;}
"while"    {return WHILE;}
"return"   {return RETURN;}
{id}       {yylval._ident = insert_id(yytext); return ID;}
{string}   {return STRING;}
"<="      {return LE;}
">="      {return GE;}
"<"       {return LT;}
">"       {return GT;}
"!="      {return NE;}
"=="      {return EQ;}
[+-]      {yylval._op = yytext[0]; return ADDOP;}
.         { return yytext[0]; }
```

图 4-2 词法分析器规则部分

其中定义的返回值在句法分析器中说明, 数字、变量使用 insert\_id 函数保存, 其他字符一律直接返回。

## 4.2 句法分析器设计

句法分析器主要参考了《基于 C-- 语言的编译原理实验教程》和老师网站上的相关代码。

### 4.2.1 输入输出设计

因为需要命令行输入文件名，输入设计类似于实验一的设计：

```
void main(int argc, char **argv)
{
    ++argv;
    --argc;
    if (argc > 0)
    {
        yyin = fopen(argv[0], "r");
    }
    else
    {
        yyin = stdin;
    }
    fun_head = (struct fun_list *) malloc(sizeof(struct fun_list));
    fun_head->next = NULL;
    yyparse();
    free(fun_head);
    printf("right!");
}
```

图 4-3 句法分析器输入设计

如果正常结束，则输出“right”。所有的错误均由 yyerror 函数处理，这里只是简单地输出错误信息，如图：

```
yyerror(char *s)
{
    printf("%s\n", s);
    exit(1);
}
```

图 4-4 yyerror 函数

### 4.2.2 token 和主要 type 设计

变量类型定义如下：

```
%union{    char *_ident;
            char _op;
            int value;
            symbol *_sym;
            expr _expr;
};
```

图 4-5 %union

其中 symbol 结构体主要使用的参数是 type（类型）和 name（名称），expr 结构体主要使用的参数是 type（类型）

token 有：

```
%token ID NUM INT VOID CHAR IF ELSE WHILE RETURN
CDECL STDCALL EQ NE LT GT LE GE AND OR
ADDOP
STRING
```

图 4-6 token-1

```
%token <_ident> NUM
%token <_ident> ID
%token <_op> ADDOP
```

图 4-7 token-2

其中，除了一些名字就可以反应内容的 token，其他的表示：

(1) EQ、NE、LT、GT、LE、GE：关系运算符，分别代表"=="、"!="、"<"、">"、"<="、">="

(2) ADDOP：既可以作为正负号，又可以作为加减号

(3) STRING：仅作为字符串常量

主要的 type 等：

```
%start   program
%right   '='
%left    ADDOP
%left    '*' '/'
```

图 4-8 type 等-1

```
%type <value> type_spec
%type <_sym> var
%type <_expr> factor
%type <_expr> unary_expr
%type <_expr> term
%type <_expr> add_expr
%type <_expr> simple_expr
%type <_expr> logicand_expr
%type <_expr> logic_expr
%type <_expr> expr
%type <_sym> fun_tag

%type <_sym> call
```

图 4-9 type 等-2

其中：

(1) 开始符是 program

(2) 右结合运算符：“=”（赋值）

(3) 左结合运算符：所有的算数运算符，包括+，-，\*，/，%。

(4) type\_spec：类型声明

(5) var：变量

(6) factor：因子（除了不可再分的 var、数字、字符串常量，还有可以递归展开的括号表达式）

(7) unary\_expr：基本式，因子添加符号和非组成

- (8) term: 项, 基本式的乘、除、模操作
- (9) add\_expr: 计算式, 项的加减
- (10) simple\_expr: 简单式, 即以关系符号链接起计算式
- (11) logicand\_expr: 逻辑与式, 简单式的与
- (12) logic\_expr: 逻辑式, 逻辑与表达式的或
- (13) expr: 式子, 包括赋值式和逻辑式
- (14) fun\_tag: 函数名
- (15) call: 函数调用

### 4.2.2 整体句法框架

整体句法框架参考附件句法规则框架.txt。

其中存在一处移进/规约错误, 但是不影响整体情况。

默认条件表达式的 if-else 在 if 之前。

### 4.2.3 标志定义问题处理

主要解决四类问题, 即变量的未定义和重定义、函数的未定义和重定义。

对于变量的未定义和重定义问题, 使用符号表来实现。

考虑到使用的语法中, 变量定义只能在全局或者在函数的开始, 这里还要考虑函数参数也应该被认为是变量定义, 在定义变量时使用符号表的 insert 函数来增加变量。例如:

```
var_declar: type_spec ID ";"
{
    struct table * tp = table_stack[table_len-1];
    symbol *pp = lookup($2, table_stack[table_len-1]);
    if(pp!=NULL)
    {
        yyerror("val redeclare!");
    }
    //printf("size of tp: %d",sizeof(*tp));
    //printf("tp->level: %d\n",tp->level);
    symbol * p = insert($2, tp);
    int width = 0;
    p->type = $1;
    p->offset = offset_stack[offset_len-1];
    if ($1 == INT) width = 4; else if ($1 == CHAR) width = 1;
    offset_stack[offset_len-1] += width;
}
;
```

图 4-10 全局变量定义



局部变量定义和函数参数定义代码相似。

考虑到每一个变量都有自己的作用域，局部变量只能作用于局部，而全局变量可以在定以后的任何地方使用，所以全局变量放在一个表中，每个函数的局部变量放在其他的表中。

定义表最多有 1000 个，并适时改变 level 值，例如程序开始和函数定义的时候：

```
fun_declar: type_spec fun_tag '(' LL_params ')' comp_stmt
{
    // $2->offset = offset_stack[table_len-1];
    if(fun_exist($2->name)!=0)
    {
        yyerror("function redeclared!\n");
    }
    // printf("fun declareing!\n");
    // printf("name:%s\n", $2->name);
    struct fun_list *p=(struct fun_list *)malloc(sizeof(struct fun_list));
    if(p==NULL)
    {
        yyerror("malloc error\n");
    }
    p->type=$1;
    p->name=$2->name;
    p->next=NULL;
    fun_insert(p);
    offset_len--;
    table_len--;
    level--;
}
```



图 4-11 level 改变示例 1

LL : {level++};

图 4-12 level 改变示例 2

这样，我们就可以来进行错误处理：

(1) 在定义变量的时候，先使用 lookup 函数查看变量是否已经定义，如果没有，则报错——变量重定义：

```
symbol *pp = lookup($2, table_stack[table_len-1]);
if(pp!=NULL)
{
    yyerror("val redeclare!");
}
```

图 4-13 变量重定义错误判断

(2) 在使用变量的时候，使用 lookup 查看变量是否定义，如果没有，则报错——变量未定义：

```
var      : ID
{
    // printf("searching ID\n");
    // printf("var ID:%s\n", $1);
    $$ = lookup($1, table_stack[table_len-1]);
    if ($$ == NULL)
    {
        yyerror("var undeclared!\n");
    }
    else
    {
        // printf("var declared!\n");
    }
}
;
```

图 4-14 变量未定义错误判断

对于函数名，考虑到函数定义不能在函数内部，所以这里可以简单地使用一条链表来存储函数的函数名和类型，添加 `fun_insert` 函数来插入新的函数信息，使用 `fun_exist` 来查看函数是否已经定义，如果已经定义，返回类型值：

```
struct fun_list
{
    char* name;
    int type;
    struct fun_list *next;
};

struct fun_list *fun_head;

int fun_exist(char *name)
{
    struct fun_list *p=fun_head->next;
    for(;p!=NULL;p=p->next)
    {
        //printf("%s vs %s\n",name,p->name);
        if(strcmp(name,p->name)==0)
        {
            //printf("match!\n");
            return p->type;
        }
    }
    return 0;
}
```

图 4-15 fun\_exist 函数

```
void fun_insert(struct fun_list* p)
{
    //printf("%s\n",p->name);
    struct fun_list* cur=fun_head;
    for(;cur->next!=NULL;cur=cur->next);
    cur->next=p;
}
```

图 4-16 fun\_insert 函数

这样，我们就可以来进行错误处理：

(1) 在定义函数时，使用 `fun_exist` 函数，如果函数已经定义，则报错——函数重定义，否则，使用 `fun_insert` 函数像函数信息链表中插入此函数信息。

```
fun_declar : type_spec fun_tag '(' LL params ')' comp_stmt
{
    //$2->offset = offset_stack[offset_len-1];
    if(fun_exist($2->name)!=0)
    {
        yyerror("function redeclared!\n");
    }
}
```

图 4-17 函数重定义错误判断

(2) 在声明函数和调用函数时，使用 `fun_exist` 函数，如果函数未定义，则

报错——函数未定义。

```
int ttype=fun_exist($2->name);
if(ttype==0)
{
    yyerror("function undeclared!\n");
}
```

图 4-18 函数未定义错误判断

#### 4.2.4 类型问题处理

这里解决的类型问题有运算类型错误、函数调用参数类型不匹配、函数声明与定义类型不相符。

对于运算类型错误，涉及到的最上级的非终结符是 `expr`，所以对所以 `expr` 可以产生的非终结符的产生式都进行类型判断，虽然比较多但逻辑都是类似的，例如：

```
simple_expr : add_expr relop add_expr
{
    if($1.type==STRING || $3.type==STRING)
    {
        yyerror("type error, relop\n");
    }
    $$type=INT;
}
| add_expr      {$$.type=$1.type;}
;
```

图 4-19 简单式判断逻辑

对于最底层的因子，有：

```
factor : '(' expr ')' {$$.type=$2.type;}
| var {$$.type=$1->type;}
| call {$$.type=$1->type;}
| NUM {$$.type=INT;}
| STRING {$$.type=STRING;}
;

call : ID '(' args ')'
{
    //printf("call ID:%s\n",$1);
    $$->type=fun_exist($1);
    if($$->type==0)
    {
        yyerror("function undeclared!\n");
    }
}
;
```

图 4-20 因子处类型赋值

对于函数调用参数类型不匹配，这里用最简单的方法，即认为函数参数可以缺省和冗余，并且认为 `int` 和 `char` 可以相互赋值，这样，调用函数时的参数只不

能是 string 类型（本次实验没有考虑指针类型和数组类型）。

```
args      : arg_list
          |
          ;

arg_list  : arg_list ',' expr
          {
            if($3.type==STRING)
            {
              yyerror("fun val error\n");
            }
          }
          | expr
          {
            if($1.type==STRING)
            {
              yyerror("fun val error\n");
            }
          }
          ;
```

图 4-21 函数调用参数类型不匹配错误判断

对于函数声明与定义类型不相符，只需要在函数声明时将函数信息链表中的类型和声明类型作比较即可。

```
| type_spec fun_tag '(' LL params ')' ';'
{
  //$2->offset = offset_stack[offset_len-1];
  int ttype=fun_exist($2->name);
  if(ttype==0)
  {
    yyerror("function undeclared!\n");
  }
  //struct fun_list *p=(struct fun_list *)malloc(sizeof(struct fun_list));
  if(ttype!=yyerror)
  {
    yyerror("fun declare type error!\n");
  }
}
;
```

图 4-22 函数声明与定义类型不相符错误判断

## 4.3 测试结果

测试文件除了给定的测试文件，还编写了其他的一些文件。语法错误不再测试。

### 4.3.1 正确的文件 s2.c

测试结果：

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s2.c
right!
```

图 4-23 s2.c 测试

### 4.3.2 变量未定义错误测试

s1.c 的情况，测试结果：

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s1.c  
var undeclared!
```

图 4-24 s1.c 测试

### 4.3.3 变量重定义错误测试

编写如下的测试程序 s4.c:

```
1 void main(void)
2 {
3     int i;
4     int j;
5     int k;
6     int i;
7     i=6;
8     i=j+4;
9     i=4+6;
10    k=6;
11 }
```

图 4-25 s4.c

测试结果：

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s4.c  
val redeclare!
```

图 4-26 s4.c 测试

### 4.3.4 函数未定义错误测试

编写如下的测试程序 s7.c:

```
1 void main(void)
2 {
3     int i;
4     int j;
5     j=6;
6     i=j+4;
7     i=4+6;
8     j=min(i, j);
9 }
```

图 4-27 s7.c

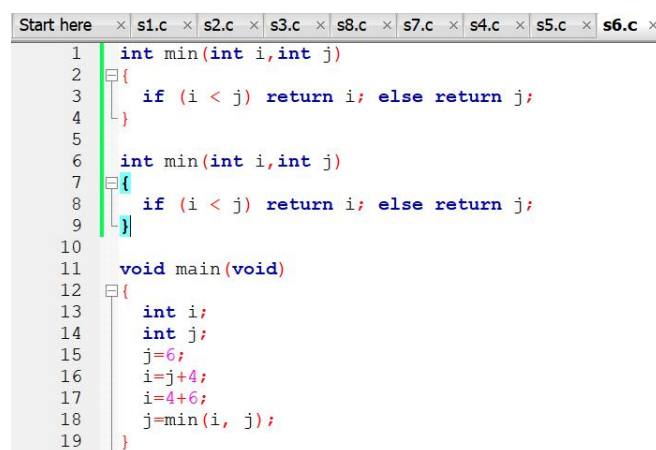
测试结果:

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s7.c
function undeclared!
```

图 4-28 s7.c 测试

### 4.3.5 函数重定义错误测试

编写如下的测试程序 s6.c:



```
Start here x s1.c x s2.c x s3.c x s8.c x s7.c x s4.c x s5.c x s6.c x
1 int min(int i,int j)
2 {
3     if (i < j) return i; else return j;
4 }
5
6 int min(int i,int j)
7 {
8     if (i < j) return i; else return j;
9 }
10
11 void main(void)
12 {
13     int i;
14     int j;
15     j=6;
16     i=j+4;
17     i=4+6;
18     j=min(i, j);
19 }
```

图 4-29 s6.c

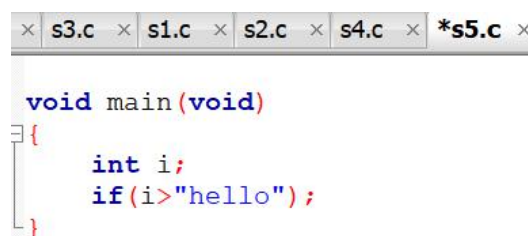
测试结果:

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s6.c
function redeclared!
```

图 4-30 s6.c 测试

### 4.3.6 运算类型错误测试

编写测试程序 s5.c, 更改文件多测几组:

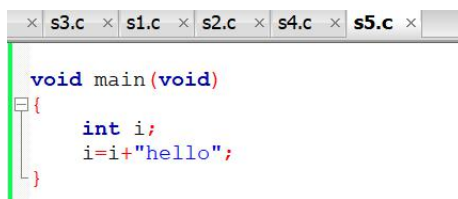


```
x s3.c x s1.c x s2.c x s4.c x *s5.c x
void main(void)
{
    int i;
    if(i>"hello");
}
```

图 4-31 s5.c-1

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s5.c
type error, relop
```

图 4-32 s5.c-1 测试

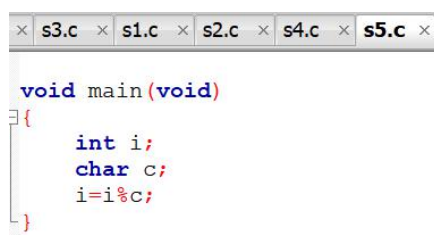


```
void main(void)
{
    int i;
    i=i+"hello";
}
```

图 4-33 s5.c-2

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s5.c
type error, addop
```

图 4-34 s5.c-2 测试



```
void main(void)
{
    int i;
    char c;
    i=i%c;
}
```

图 4-35 s5.c-3

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s5.c
type error, mulop
```

图 4-36 s5.c-3 测试

### 4.3.7 函数调用参数类型不匹配错误测试

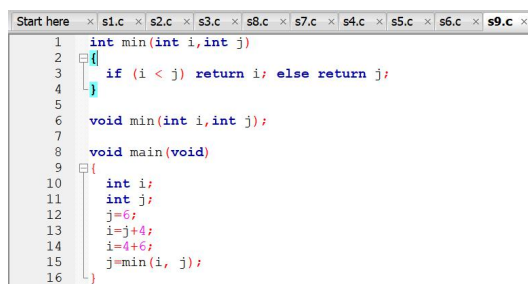
使用测试文件 s3.c，测试结果：

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s3.c
fun val error
```

图 4-37 s3.c 测试

### 4.3.8 函数声明与定义类型不相符错误测试

编写如下的测试程序 s9.c:



```
1 int min(int i,int j)
2 {
3     if (i < j) return i; else return j;
4 }
5
6 void min(int i,int j);
7
8 void main(void)
9 {
10     int i;
11     int j;
12     j=6;
13     i=j+4;
14     i=4+6;
15     j=min(i, j);
16 }
```

图 4-38 s9.c



测试结果:

```
PS D:\my_world\class\大三\大三下\主修课\语言分析技术\实验\3\文件> ./y.tab.exe s9.c
fun declare type error!
```

图 4-39 s9.c 测试

#### 4.4 程序不足和可以改进的地方

- (1) 函数参数判定可以更加精准，即不允许缺省和冗余
- (2) 可以对变量长度做出限定
- (3) 测试的程序比较小型化，可能存在其他错误

## 五、遇到的问题与解决

本次实验碰到的问题还是很多的，包括很多 C 语言部分的错误和产生式的各种错误，其中最麻烦的一个就是使用 `malloc` 申请空间之后，某个不相关的变量值被莫名其妙地改掉了导致判定函数重定义和未定义错误时总是出现错误，如图：

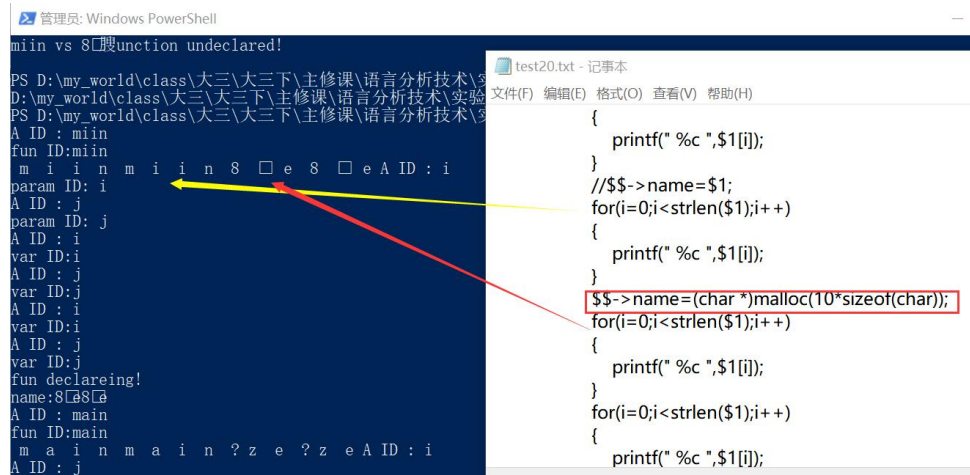


图 5-1 malloc 错误

这可能是我是用的 IDE——code:block 的问题，之前也挺一些朋友也提到过 `malloc` 可能会改到其他变量的值。最终我放弃了 `malloc` 的使用，使用一个 `char` 数组来解决问题。

本次实验花的时间比较久，文件也是改过来改过去，很多代码是没用的，但是因为太杂乱没有时间去整理清楚。